

AA-02

Backend Extension: Modules, Namespaces and Tenancy

Eleven modules, four API namespaces, and how tenant scope is made unforgeable

30 August 2026 · Rev 1.0 — baseline

MODULES Eleven	API NAMESPACES Four	IDEMPOTENT ROUTES 70
ERROR CODES Fourteen	CRON WORKFLOWS Eleven	RECURRING COST Rs 0

This document is about the inside of one process. atavishaala-api already exists, already serves the mobile app, and already holds the only database credential in the programme; what follows is how it absorbs three portals and two new principal kinds without becoming four services and without letting one supplier read another supplier row. Sections 2.4 and 2.5 are the load-bearing pair: tenancy is not a filter a developer remembers to write, it is a claim in a verified token and a repository wrapper that cannot be bypassed. Section 2.7 is the honest one — the scheduler is GitHub Actions cron, it is late sometimes, and every deadline in the system is therefore computed from a stored timestamp.

2.1 THE SHAPE OF THE API REPO

One process, eleven modules, and three rejected ways of splitting it

atavishaala-api is a modular monolith on Cloud Run in asia-south1 and it stays one. The three portals add 164 authenticated endpoints across three namespaces, two new principal kinds, 36 tables and a permission matrix, and none of that changes the deployment unit. The boundaries that matter are enforced by the compiler and by ESLint inside one build, not by a network hop between two builds.

That is a decision rather than an inheritance, so it is argued below. The alternatives were not strawmen — each is what a competent engineer would propose, and each lost for a reason specific to this stack and this budget rather than to microservices in general. AA-00 section 0.4 states the runtime topology this sits inside; AA-00 section 0.6 states the fourteen invariants that the module boundaries exist to make enforceable.

#	REJECTED ALTERNATIVE	WHY IT LOST
A	A separate admin service. atavishaala-admin-api as its own Cloud Run service, its own image, its own deploy, holding the backoffice, partners and fleet modules; the existing api keeps the app surface.	Two reasons, and the first is structural. That service needs its own DATABASE_URL, and the invariant the whole programme rests on is that exactly one process writes Postgres — I-01 phrases it as one backend and three thin clients, and the reason it holds is that there is physically one credential. Two writers means two Prisma clients, two migration histories to keep in step, and two places where a status column can be written without going through assertTransition (I-05). Second, on the always-free tier the cost is cold starts: each Cloud Run service scales to zero independently, so a dispatcher opening the admin portal pays one cold start and a rider accepting an offer pays another, and a request that crosses from admin to orders pays both. Doubling the cold-start surface to gain a deploy boundary we do not need is the wrong trade at four developers.
B	Serverless functions per endpoint. Cloud Functions or Cloudflare Workers, one handler per route, 164 independently deployed units and no long-lived process at all.	Prisma against Neon collapses under this shape. Each invocation constructs its own client and therefore its own connection, and the free Neon tier gives a small connection ceiling; a burst of ten dispatchers refreshing a job list exhausts it and every subsequent request fails on connection acquisition rather than on anything a test would have caught. The available fix is the Neon serverless driver over HTTP plus a pooler endpoint, which means giving up transactions that span statements — and the manifest disclosure in I-07 and every ledger write in I-04 are defined as single transactions. Losing multi-statement transactions to gain per-route scaling is not a trade this system can make.
C	A BFF per portal. Three thin Node services shaping responses for admin, partner and fleet, each calling the one api underneath.	It adds a hop and, worse, a second place to get authorisation wrong. If a BFF holds a service token to reach the api, then the api is no longer evaluating the real principal and the permission check has moved into the layer with no audit table behind it; if the BFF forwards the user token unchanged, it is a proxy and earns nothing. The response-shaping it would buy is already bought by the generated contract package in AA-01 and by the masking extension running in the serialiser, both of which live in the api where the data is.

2.1.1 The directory layout

Two top-level directories carry everything. src/modules holds the eleven business modules of section 2.2, each with the same five files and its own test folder. src/platform holds the six cross-cutting concerns that every module consumes and no module owns.

atavishaala-api/	
prisma/schema.prisma	one schema, one migration history, one client
prisma/migrations/	M1..M8, forward-only (AA-03)
src/index.ts	express 5 app, namespace mounting, nothing else
src/modules/<name>/	
router.ts	route table + zod bindings + route metadata (meta.money etc.)
service.ts	business rules, transactions, the only caller of repo.ts
repo.ts	prisma access; in partner/fleet paths only via scoped()
schema.ts	zod request/response schemas; source of @atavishaala/contract
policy.ts	permission constants + step-up flags for this module
types.ts	the module's public types — the ONLY cross-module import
fsm.ts	transition table (orders, shipments, jobs, kyc, payouts)
__tests__/	unit on pure functions, integration on the router
src/platform/	
auth/	jwt verify, requireActor, requirePermission, requireFreshFactor
pii/	the masking client extension, last4(), discloseOrThrow()
money/	paise arithmetic, ledger append, no float anywhere
queue/	job_queue enqueue + drain (FOR UPDATE SKIP LOCKED)
audit/	admin-router middleware, before/after diff writer
idem/	idempotency_keys middleware, replay + mismatch detection
packages/shared/	errors.ts, money.ts, permissions union, contract generator

The uniformity is the point. A developer opening `src/modules/fleet` knows that the route table is in `router.ts`, that nothing outside `service.ts` touches the database, and that a permission string is in `policy.ts` and nowhere else. In week 6 that saves more time than any amount of cleverness in the layout would.

2.1.2 The import rule, and the rule that enforces it

A module may import from `src/platform` freely, and from another module it may import `types.ts` and nothing else. It may not import another module's `repo.ts`, `service.ts` or `router.ts`. So orders may read the `Shipment` and `PartnerOrgRef` types from `partners/types.ts`, but if it needs a partner's rate card it calls a function the `partners` module exports through its own `service` and re-exports in `types.ts` as a signature — it does not reach into `partner_rate_cards` itself.

The reason is ownership of writes. If two modules can write the same table, the transition table in `fsm.ts` stops being authoritative and I-05 becomes unenforceable, because the guard lives in the owning module. Restricting the import to `types.ts` means a cross-module write is a compile error rather than a code-review observation.

THE BOUNDARY RULE: ESLINT-PLUGIN-BOUNDARIES, ZONE-BASED

Every path under `src/modules` is tagged as a module element type and `src/platform` as `platform`. The `boundaries/element-types` rule allows modules to depend on `platform` and on `shared`, and allows module-to-module only where the imported path matches the `types.ts` file; every other cross-module edge is an error. It is paired with `import/no-restricted-paths` as a second expression of the same constraint so that a tag misconfiguration does not silently open the door. Both run in the lint gate of AA-01 section 1.7 alongside the eight custom `aa/*` rules, of which the tenancy-specific ones are set out in section 2.5.

2.2 THE ELEVEN MODULES

Three new, four extended, one rewritten, three untouched

MODULE	OWNER	STATUS	TABLES	NAMESPACES	RESPONSIBILITY
identity	BE-1	Existing	users, otp_challenges, refresh_families, sessions	app, public	Customer OTP login, refresh rotation, family revoke.
users	BE-1	Existing	user_profiles, addresses, wishlists	app, admin	Customer profile, saved addresses, language preference.
catalog	BE-1	Extended	categories, products, product_variants, product_media, brands	app, admin, partner	Adds partner-authored draft products and the moderation queue.
advisor	BE-2	Extended	advisory_requests, advisory_media, advisory_notes, expert_shifts	app, admin	Adds the agronomist console: queue claim, SLA clock, callback log.
commerce	BE-2	Existing	carts, cart_items, coupons, pricing_snapshots	app, admin	Cart, pricing, coupon validation. Unchanged.
orders	BE-2	Rewritten	orders, order_items, shipments, shipment_items, order_events	app, admin, partner, fleet	Order splits into shipments per supplier. Order status is derived.
money	BE-2	Extended	payments, refunds, wallet_ledger, wallet_accounts, referrals	app, admin	Unchanged customer money. Adds refund approval workflow for staff.
platform	BE-1	Extended	settings, feature_flags, job_queue, notifications, audit_log	admin	Adds audit_log with before/after diff and the pii_disclosures register.
partners	BE-1	NEW	partner_orgs, partner_users, partner_kyc, partner_roles, partner_rate_cards, partner_payouts	admin, partner	Product-company onboarding, KYC verification, commission, weekly settlement.
fleet	BE-2	NEW	fleet_orgs, fleet_users, fleet_kyc, fleet_roles, fleet_service_areas, riders, vehicles, delivery_jobs, job_offers, pod_records, cod_remittances, fleet_payouts	admin, fleet	Delivery-company onboarding, offer allocation, POD, COD reconciliation.
backoffice	BE-1	NEW	staff_users, staff_roles, role_permissions, staff_totp, saved_views, exports	admin	Staff identity, the 88-permission matrix, saved list views, async CSV export jobs.

The status column is the schedule. Three modules are **NEW** — `partners`, `fleet` and `backoffice` — and they are where the work is: they own no existing behaviour, they can be built against a clean transition table, and a bug in them cannot reach the mobile app because no app route touches them. BE-1 takes `partners` and `backoffice`, BE-2 takes `fleet`, and AA-11 sequences them from week 2.

Four are Extended and one is Rewritten, and those five are the risky set. Extending a module means adding to a file that a paying customer is already exercising, so the cost is not the new code — it is the regression surface underneath it.

2.2.1 What "extended" costs: the app contract is frozen

The rule for every Extended module is that the mobile contract stays byte-compatible. Not source-compatible, not semantically-equivalent: byte-compatible. A response body served to `atv_app` before this programme must serialise to the same bytes after it, with the same field names, the same field order, the same types and the same absent fields absent. The reason is that there is a React Native app in the field which we do not control the update cadence of; a user on an old build who does not update for six weeks is a user whose parser must keep working.

The operational consequence is that `/api/v1/app` is a closed set. Additions go to new paths in the new namespaces, never to an existing app route. When the fleet module needs the shipment to expose a rider name, it does not add `rider_name` to the app's shipment payload; it adds it to `/api/v1/admin` and `/api/v1/fleet`, and the app tracker reads what it already read. Where the app genuinely must show something new — the delivery tracker gaining shipment-level granularity — it reads `order_events`, which is an append-only stream it was already consuming, so the shape of the event is new but the endpoint is not. Section 2.8 covers why that choice also solved the derived order status problem.

catalog is the awkward one. It gains partner-authored draft products, which means the products table gains a `partner_org_id` and a moderation state, and every existing customer-facing query must therefore learn to exclude anything not APPROVED. That is a change to a query the app depends on, in the one place where getting it wrong publishes an unreviewed product to customers. The mitigation is that the filter lives in one repository function which every catalog read path calls, and the contract sweep in AA-01 section 1.7 asserts the app's catalog responses are byte-identical against recorded fixtures.

WHY ORDERS IS A REWRITE AND NOT AN EXTENSION

The existing orders module assumed one order equals one fulfilment. Multi-supplier fulfilment breaks that assumption at the root: an order now owns a set of shipments, each with its own supplier, its own sixteen-state machine and its own delivery job. You cannot reach that from the old model by addition, because the old model wrote `orders.status` directly and the new one derives it. So `orders/fsm.ts` is new code with a new transition table, and the compatibility obligation is discharged at the serialiser rather than in the data model.

WHY COMMERCE, IDENTITY AND USERS ARE UNTOUCHED

Cart, pricing, coupons, OTP login, refresh rotation, profiles and addresses are complete and correct for this programme. No portal needs to change how a customer logs in or how a cart prices. Naming them Existing here is a commitment, not an observation: a pull request that edits `src/modules/commerce` during these ten weeks needs a stated reason in the description, because the most likely explanation is that a new requirement was put in the wrong module.

2.3 THE FOUR API NAMESPACES

One namespace per audience, plus the unauthenticated one

PATH PREFIX	AUDIENCE CLAIM	PRINCIPAL	CONTENTS
<code>/api/v1/public</code>	none	anonymous	Health, legal docs, pincode serviceability, OTP request.
<code>/api/v1/app</code>	<code>atv_app</code>	customer	The React Native app. Existing surface, frozen shape.
<code>/api/v1/admin</code>	<code>atv_admin</code>	staff	Admin portal. Widest permission surface, fully audited.
<code>/api/v1/partner</code>	<code>atv_partner</code>	partner	Supplier portal. Every query tenant-scoped to <code>partner_org_id</code> .
<code>/api/v1/fleet</code>	<code>atv_fleet</code>	fleet	Fleet portal. Every query tenant-scoped to <code>fleet_org_id</code> .

Four of those five are authenticated and each corresponds to exactly one token audience, which is invariant I-02: a token minted for one audience is rejected on every other namespace. The fifth, `/api/v1/public`, carries no principal at all and holds only health, legal documents, pincode serviceability and the OTP request — anything an anonymous caller may reach is in that prefix, and a route's prefix is therefore a statement about who may call it that can be read without opening the handler.

The prefix is not cosmetic routing. Each namespace is a separate Express router with its own middleware stack, its own rate-limit bucket and, for admin, its own audit middleware mounted at the router rather than per handler (I-08), so a newly added admin endpoint cannot forget to be audited. AA-04 is the endpoint list itself; this section

is only the chain each of those 164 endpoints runs through before its handler sees a request.

2.3.1 The middleware chain, in order

1. requestId — read X-Request-Id if present and well-formed, otherwise mint a UUID v4. Bound to the response header and to every log line for the life of the request.
2. Pino child logger bound with the request id, the actor kind and the tenant id, constructed with the redaction paths of section 2.10 already applied. Nothing logs outside this child logger.
3. Body size limit — 256 KB for JSON. Uploads never pass through the api: the portal receives a signed Cloudinary URL and posts direct, so there is no multipart path to size.
4. CORS against ALLOWED_ORIGINS as an exact list. No wildcard, no suffix match, no reflected Origin. A non-listed Origin is refused here, before anything reads the body.
5. Rate limit, bucketed by namespace and then by principal id, with the per-namespace numbers in section 2.10. The bucket is chosen by prefix, so a rider burst cannot spend the admin budget.
6. JWT verify — RS256 only, with the algorithm pinned in code rather than read from the header, issuer and audience checked, JWKS held in memory with a 10-minute cache and a single-flight refetch so a key rotation does not stampede.
7. requireActor(kind) — assert the verified aud claim equals the namespace audience. On mismatch: 401 ACTOR_MISMATCH, and stop. Nothing below this line runs.
8. Tenant scope extraction — read partner_org_id or fleet_org_id from the claim, reject with 400 SCOPE_IN_REQUEST if the input carries either (section 2.4), and bind the scope to the request context where scoped() will find it.
9. requirePermission(...) against the effective permission set, DENY winning over ALLOW. Admin and portal-org roles only; the app namespace has no permission layer at all.
10. requireFreshFactor(300) on the routes marked for step-up: a TOTP factor verified within the last 300 seconds. Failure is 401 STEP_UP_REQUIRED carrying a challenge hint.
11. Idempotency — on a route tagged meta.money or meta.offer, look up (actor, route, key) and either replay, reject a body mismatch, or reserve the key for this attempt (section 2.6).
12. Zod parse of params, query and body against the module schema.ts. Unknown keys are stripped rather than tolerated. Failure is 400 VALIDATION_FAILED with a field path.
13. The handler. It receives a typed request, a scoped repository and a bound logger, and it may assume the principal, the tenant and the permission are already settled.
14. Response serialiser, which runs the PII masking extension unconditionally (I-06), then the audit writer for any non-GET on /api/v1/admin (I-08), then the idempotency recorder.

2.3.2 Why requireActor sits before requirePermission

Step 7 could plausibly be merged into step 9 — a permission check that fails also refuses the request, so why two gates. Because they answer different questions and only one of them is safe to answer out loud. requireActor asks whether this kind of principal belongs on this namespace at all; requirePermission asks whether this particular principal may perform this operation.

Take an atv_partner token presented to POST /admin/payout-runs/:id/release. If that request reached permission evaluation, the outcome would be 403 PERMISSION_DENIED — and a 403 is a statement that the route exists, accepts this method, and is guarded by a permission this caller happens to lack. That is a free map of the admin surface, handed to a supplier who is already authenticated and therefore already past the rate limiter's anonymous bucket. So the response is 401 ACTOR_MISMATCH instead: the audience is wrong, the request never reaches the router's permission layer, and the answer is identical for a real admin path and for a path that does not exist. Enumerating the admin surface with a partner token yields one response code and no information.

The second reason is enforceability. Permission strings are per-namespace by construction — payout.run.release exists only in the admin union, p.shipment.accept only in the partner union — so a cross-audience request is a lookup against the wrong union, and an implementation that handles that gracefully is an implementation that has to decide what a missing permission means. Refusing earlier removes the question. The contract sweep in AA-01 section 1.7 asserts it directly: every non-public route is called with the three wrong audiences and must answer 401.

2.4 TENANCY

Invariant I-03: the scope is a claim, not a parameter

I-03	TENANT SCOPE COMES FROM THE TOKEN
The rule	partner_org_id and fleet_org_id are read from the verified token claim only. A request body or query string that carries either is rejected with 400 SCOPE_IN_REQUEST.
Enforcement	ESLint rule aa/no-tenant-in-input plus a repo-layer scoped() wrapper that is the only permitted entry point in partner and fleet namespaces.

Two of the four authenticated namespaces are multi-tenant. Every row a partner principal may read or write belongs to one partner_orgs row, and every row a fleet principal may touch belongs to one fleet_orgs row. There is no cross-tenant read in the product: a supplier does not see another supplier's prices, a fleet does not see another fleet's riders, and no legitimate screen in either portal needs to.

That makes tenancy the highest-consequence, lowest-visibility rule in the backend. A missing WHERE clause does not throw, does not log, and does not fail a test written against a database with one seeded organisation. It returns a page of rows that looks entirely plausible. So the defence is not care — it is that the scope has exactly one source and the repository layer cannot be called without it.

2.4.1 The claim shape

```
// Access token, atv_partner. RS256, ACCESS_TTL_SECONDS. Verified before any of this is read.
{
  "iss": "https://api.atavishaala.in",    "aud": "atv_partner",
  "sub": "pu_<partner_user_id>",         "jti": "<uuid v4>",
  "partner_org_id": "po_<ulid>",         // THE tenant scope. Present exactly once.
  "roles": ["PARTNER_OPS"],              // resolved to permissions server-side, not trusted
  "iat": 1756531200, "exp": 1756532100
}

// Access token, atv_fleet. Same shape, different scope key and a rider-seat marker.
{
  "iss": "https://api.atavishaala.in",    "aud": "atv_fleet",
  "sub": "fu_<fleet_user_id>",            "jti": "<uuid v4>",
  "fleet_org_id": "fo_<ulid>",            // THE tenant scope.
  "rider_id": "rd_<ulid>" | null,         // set only for a rider seat; null for dispatch
  "roles": ["FLEET_DISPATCH"],
  "iat": 1756531200, "exp": 1756532100
}
```

The scope key appears in exactly one place: the verified claim. It is written by the token minter at login from the partner_users or fleet_users row, it is signed, and the middleware reads it after signature verification and never before. A rider seat additionally carries rider_id, which narrows within the tenant but never widens across it — a rider token is a fleet token with less reach, not a different kind of scope.

2.4.2 A request that carries its own scope is rejected, not sanitised

If partner_org_id or fleet_org_id appears in a request body, a query string or a path parameter on any route, the request is refused with 400 SCOPE_IN_REQUEST at step 8 of the chain, before Zod runs and before the handler exists. The same check covers the obvious synonyms — org_id, orgId, partnerOrgId, tenant_id — and the ESLint rule aa/no-tenant-in-input makes the corresponding Zod schema impossible to write in the first place: a schema under a partner or fleet route that declares either field is a lint error, so the field cannot be accepted even accidentally.

The instinct is to ignore the value rather than reject the request, because ignoring it is obviously safe: the scope still comes from the claim, so nothing bad happens. That instinct is wrong for two reasons and both are about what happens afterwards.

First, silent ignoring lets a client believe it is scoping when it is not. A front-end developer who sends partner_org_id and gets sensible results has just formed a false model of the API, and the day someone builds an internal tool that iterates organisations on that model, it returns one organisation's data labelled as many. Rejecting turns that misunderstanding into a 400 on the first request, which is the cheapest possible moment to

correct it. Second, a request carrying a tenant id it was not given is the signature of an escalation attempt, and ignoring it destroys the only evidence. The rejection writes a security event — actor, tenant from claim, tenant attempted, route, request id — at warn level, and repeated events from one principal are what a weekly review can actually act on. A 200 leaves nothing to review.

THERE IS ONE EXCEPTION, AND IT IS IN THE OTHER DIRECTION

Admin routes take an organisation id in the path, because a staff principal is legitimately cross-tenant: GET /admin/partners/:id is the whole point of the admin portal. Those routes are not scoped by claim, they are guarded by permission and audited on every mutation (I-08). The distinction is namespace-shaped rather than parameter-shaped: an id in the path is data on /admin, and a 400 on /partner and /fleet.

WHAT THIS DOES NOT DEFEND AGAINST

A partner user who has genuinely been invited into two organisations holds two tokens and may read both, correctly. A compromised partner_users row lets an attacker act as that tenant fully. Tenancy bounds blast radius across organisations; it does nothing within one. AA-08 covers session revocation and the refresh-family kill switch, which are the controls for that case.

2.5 SCOPED(): THE ONLY REPOSITORY ENTRY POINT

A Prisma client extension, four rules, and a 404 that is deliberate

Reading the scope from the claim solves half the problem. The other half is that every query in the partner and fleet namespaces has to use it, and "every" is not a property a code review can establish once and rely on for ten weeks. So the scope is not passed to queries; it is bound to a client and the queries cannot be written without it.

scoped(orgId, kind) returns a Prisma client extension over the tenant-owned models. Any read or write issued through it has the tenant predicate merged into its where clause by the extension, and a delegate whose model is tenant-owned is unreachable except through it.

```
// src/platform/tenancy/scoped.ts (shape, not the full file)
const PARTNER_OWNED = ['partnerUser', 'partnerKyc', 'partnerRateCard', 'partnerPayout',
  'product', 'productVariant', 'shipment'] as const;
const FLEET_OWNED = ['fleetUser', 'fleetKyc', 'fleetServiceArea', 'rider', 'vehicle',
  'deliveryJob', 'jobOffer', 'podRecord', 'codRemittance', 'fleetPayout'] as const;

export function scoped(orgId: string, kind: 'partner' | 'fleet') {
  if (!orgId) throw new ScopeMissingError(); // 500. Never a permissive default.
  const col = kind === 'partner' ? 'partner_org_id' : 'fleet_org_id';
  const models = kind === 'partner' ? PARTNER_OWNED : FLEET_OWNED;
  return prisma.$extends({
    query: Object.fromEntries(models.map((m) => [m, {
      async $allOperations({ operation, args, query }) {
        switch (operation) {
          case 'findMany': case 'findFirst': case 'findUnique':
          case 'update': case 'updateMany':
          case 'delete': case 'deleteMany': case 'count': case 'aggregate':
            args.where = { AND: [args.where ?? {}, { [col]: orgId }] };
            break;
          case 'create':
            args.data = { ...args.data, [col]: orgId }; // cannot be overridden by input
            break;
          default:
            throw new UnscopedOperationError(operation); // upsert, createMany, raw: not allowed
        }
        return query(args);
      },
    }]),
  }));
}
```

findUnique is included deliberately even though it looks safe: a lookup by primary key with no tenant predicate is exactly how a cross-tenant read happens, because the id came from somewhere and nothing about it says which organisation owns it. Merging the predicate turns it into a query that returns nothing rather than a query that returns another tenant's row. upsert and createMany are refused rather than rewritten: their argument shapes make a correct merge ambiguous, and an ambiguous rewrite in this layer is worse than a compile-time absence.

2.5.1 The four rules around it

#	RULE	WHY, AND WHAT ENFORCES IT
1	No bare prisma. reference for a tenant-owned model inside modules/*/repo.ts. The scoped client arrives as an argument; the module never constructs one.	An extension only helps if it is on the path actually taken. A single prisma.rider.findMany in a repo function bypasses everything above. A custom ESLint rule flags any member expression on the prisma identifier whose model is in the tenant-owned list, anywhere under src/modules, and it has no suppression path — a comment disabling it fails the lint gate. Admin repositories are exempt by construction because they live behind permission checks and audit, and the rule is scoped to the partner and fleet route trees.
2	A cross-tenant read returns 404 NOT_FOUND, never 403. Wrong tenant and non-existent are the same answer.	403 confirms that the id exists. Given sequential or guessable ids that turns a permission error into an enumeration oracle: an attacker learns how many riders a competitor has, or that a particular shipment id is real. Because the extension merges the predicate, this falls out for free rather than being remembered — the row simply is not in the result set, so the handler takes its ordinary not-found branch. The property is asserted by an integration test per tenant-owned model with two seeded organisations.
3	A missing scope claim is a 500, not a permissive query. scoped() throws before it builds a client if orgId is falsy.	The alternative is a default, and every default here is wrong. Defaulting to no predicate returns every tenant's rows; defaulting to a sentinel returns nothing and looks like an empty account. Both are silent. A ScopeMissingError is a 500 with a request id, which means the token minter or the middleware is broken and someone finds out within one request instead of after a week of odd support tickets. A 500 for a bug is correct: it is not the caller's fault and it is not the caller's problem to work around.
4	\$queryRaw and \$executeRaw are banned outside src/platform, with a named exception list reviewed by BE-1.	Raw SQL cannot be extended, so it silently escapes the tenant predicate. There are exactly three legitimate cases and all three are tenant-free: the FOR UPDATE SKIP LOCKED claim in platform/queue, the advisory-lock acquisition in the payout assembler, and the ANALYZE in the migration rehearsal. Each is a named export in platform with the reason in a header comment, and the ESLint rule allow-lists those three file paths and nothing else. A fourth case is a conversation, not a commit.

WHAT SCOPED() IS NOT

It is not row-level security. Postgres RLS would enforce this in the database and would survive a raw query, which is strictly stronger. It was not chosen because RLS needs a per-request SET LOCAL of a session variable inside the transaction, and with a pooled connection under a scale-to-zero process the failure mode of a missed SET LOCAL is a query with no policy context — which is either everything or nothing depending on the policy, and hard to test on the free tier. The application-layer choice is the weaker mechanism honestly labelled. Its whole strength is that the unsafe call is a lint error rather than a runtime hope, and rule 4 exists because raw SQL is the one hole the mechanism genuinely cannot close.

2.6 IDEMPOTENCY

Invariant I-12 across 70 of 164 authenticated routes

70 routes require an Idempotency-Key header. That is every POST or PUT that moves money, changes a state machine with an external side effect, or allocates a scarce thing. The table below is the complete set, grouped by namespace and route group; AA-04 carries each one with its permission and its request shape.

NS	GROUP	N	ROUTES REQUIRING Idempotency-Key
admin	Catalog	1	POST /admin/variants/:id/stock
admin	Orders	4	POST /admin/orders/:id/cancel, POST /admin/shipments/:id/resplit, POST /admin/shipments/:id/transition, POST /admin/returns/:id/decide
admin	Delivery	3	POST /admin/jobs/:id/assign, POST /admin/jobs/:id/rebroadcast, POST /admin/jobs/:id/cancel
admin	Money	5	POST /admin/refunds, POST /admin/refunds/:id/approve, POST /admin/wallet/:userId/credit, POST /admin/wallet/:userId/debit, POST /admin/coupons
admin	Payouts	6	POST /admin/payout-runs, POST /admin/payout-runs/:id/lock, POST /admin/payout-runs/:id/release, POST /admin/payout-lines/:id/withhold, POST /admin/cod/:id/reconcile, PUT /admin/partners/:id/rate-card
admin	Partners	4	POST /admin/partners, POST /admin/partners/:id/status, POST /admin/partner-kyc/:id/review, POST /admin/partner-bank/:id/verify
admin	Fleets	5	POST /admin/fleets, POST /admin/fleets/:id/status, POST /admin/fleet-kyc/:id/review, POST /admin/fleet-areas/:id/approve, POST /admin/riders/:id/suspend
admin	Advisory	3	POST /admin/advisory/:id/claim, POST /admin/advisory/:id/reassign, POST /admin/advisory/:id/sla
admin	Platform	6	POST /admin/customers/:id/status, POST /admin/staff, PUT /admin/staff/:id/roles, PUT /admin/roles/:id/permissions, PUT /admin/settings/:key, POST /admin/exports
partner	Catalog	4	POST /partner/products, POST /partner/products/:id/submit, PATCH /partner/variants/:id/stock, POST /partner/variants/:id/price-request
partner	Fulfilment	5	POST /partner/shipments/:id/accept, POST /partner/shipments/:id/pack, POST /partner/shipments/:id/ready, POST /partner/shipments/:id/short-ship, POST /partner/shipments/:id/handover

NS	GROUP	N	ROUTES REQUIRING Idempotency-Key
partner	Returns	2	POST /partner/returns/:id/receive, POST /partner/returns/:id/dispute
partner	Finance	1	PUT /partner/bank
partner	Org	3	POST /partner/users, POST /partner/users/:id/disable, POST /partner/kyc
fleet	Session	1	POST /fleet/auth/otp/request
fleet	Offers	2	POST /fleet/offers/:id/accept, POST /fleet/offers/:id/decline
fleet	Jobs	7	POST /fleet/jobs/:id/assign, POST /fleet/jobs/:id/pickup, POST /fleet/jobs/:id/transit, POST /fleet/jobs/:id/out-for-delivery, POST /fleet/jobs/:id/pod, POST /fleet/jobs/:id/fail, POST /fleet/jobs/:id/rto
fleet	Fleet	4	POST /fleet/riders, POST /fleet/riders/:id/documents, POST /fleet/vehicles, POST /fleet/areas/request
fleet	Finance	2	POST /fleet/cod/:id/declare, PUT /fleet/bank
fleet	Org	2	POST /fleet/users, POST /fleet/kyc

The remaining routes are exempt for one of two reasons. A GET is naturally idempotent and needs no key. A PATCH or PUT that sets an absolute value — a profile field, a saved view — converges on replay because the second write produces the same row as the first. What is never exempt is a write whose effect depends on the prior state: a signed stock delta, an append to a ledger, a transition, an allocation.

2.6.1 The mechanism

STEP	BEHAVIOUR
Key identity	A row in <code>idempotency_keys</code> with a unique index on <code>(actor_kind, actor_id, endpoint, key)</code> . The key is scoped to the actor and the route, never global: two suppliers may legitimately mint the same client-side UUID, and one supplier reusing a key on a different route is a different intent. TTL is 24 hours; the <code>idempotency-gc</code> cron deletes rows past <code>expires_at</code> .
First attempt	The middleware inserts the row in state <code>IN_FLIGHT</code> inside the same transaction the handler will use. A second concurrent request on the same key hits the unique index, gets a constraint violation rather than a race, and is answered 409 with a retry hint.
Completion	On a 2xx the row is updated once with the response status, the serialised body, and a SHA-256 of the request body. This is the one place a row is updated rather than appended, and it is not a ledger.
Replay	A repeat with the same key and the same body hash returns the stored status and the stored bytes, with <code>Idempotent-Replay: true</code> on the response. The handler does not run. Byte equality is asserted by CI, not merely a 200 — a replay that returns a different UTR or a different payout id is a duplicate write wearing a success code.
Mismatch	The same key with a different body hash is 409 <code>IDEMPOTENCY_MISMATCH</code> . The client has reused a key for a different intent, which means its key derivation is wrong, and answering with either response would be a guess about which one it wanted.
Failure	A 5xx clears the row so the operation can be retried with the same key. A 4xx stores the response like any other outcome: a validation failure is a settled answer and replaying it is correct.

WHY OFFER ACCEPTANCE NEEDS THIS AS BADLY AS A PAYMENT DOES

A payment retried twice charges a card twice, and everyone already believes that matters. POST /fleet/offers/:id/accept is the same class of operation and it is easier to get wrong. A dispatcher on a patchy mobile network taps Accept; the request reaches the api and commits; the response is lost on the way back; the portal retries. Without a key, the second accept either creates a second `delivery_jobs` assignment for one shipment or answers 409 `OFFER_TAKEN` — and `OFFER_TAKEN` is indistinguishable, from the dispatcher's side, from having lost the job to a competing fleet. So the honest client behaviour on a lost response would be to show "this job went to someone else" for a job it actually won, and the rider sits idle while the shipment shows as assigned to them. With the key, the retry is a replay: the same job id, the same assignment, the same 200. Section 2.7 explains why the wave TTLs make this a common case rather than a rare one.

2.7 THE JOB QUEUE AND THE CRON THAT DRIVES IT

Postgres SKIP LOCKED, GitHub Actions as the clock, and why that is a workaround

Cloud Run scales to zero. There is no always-on process, so there is no background worker, so anything that must happen without a user asking for it needs an external clock. The queue itself already exists and is unchanged: platform/queue enqueues into the job_queue table, and a drain claims work with `SELECT ... FOR UPDATE SKIP LOCKED LIMIT n` inside a transaction, which gives exactly-once claiming across concurrent drains without a broker. What this programme adds is more kinds of work — export jobs, webhook retries, payout accrual — and ten more scheduled entry points.

Each entry point is a GitHub Actions scheduled workflow that makes one authenticated HTTP call to a cron route, presenting `CRON_SHARED_SECRET` in a header; the route enqueues or drains and returns. Actions is the clock, not the worker: the work runs in Cloud Run where the database credential is, and a workflow that cannot reach the api fails loudly in the Actions log. AA-09 covers the operational side — what alerts, what the retry policy is, and what a missed run looks like.

SCHEDULE	WORKFLOW	WHAT IT DOES
/ * * *	offer-sweep	Expire lapsed job_offers, advance to the next wave, or mark UNASSIGNED.
/ * * *	job-queue-drain	Existing worker. Now also handles export jobs and webhook retries.
/ * * *	sla-watch	Advisory SLA breach alerts and shipment ready_by breach alerts.
0 * * * *	stale-shipment-alert	Shipments in AWAITING_ACCEPTANCE for more than 4 business hours.
30 2 * * *	accrual	Accrue partner_payouts and fleet_payouts for the previous day's deliveries.
0 3 * * *	doc-expiry-watch	Rider licence and vehicle insurance expiring within 30 days.
0 4 * * 1	payout-draft	Assemble the DRAFT partner and fleet runs for the previous week.
0 5 * * 1	cod-period-open	Open the new COD period and close declarations on the previous one.
0 6 * * *	free-tier-probe	Read usage from each provider where an API exists; alert at threshold.
0 7 1 * *	audit-archive	Export audit_log older than 180 days to R2 as gzipped NDJSON, then delete.
15 */6 * * *	idempotency-gc	Delete idempotency_keys past expires_at.

2.7.1 The honest part: this is not a scheduler

The offer-sweep row says every two minutes. It will not run every two minutes. GitHub scheduled workflows are best-effort on the shared runner pool: the documented behaviour is that a schedule may be delayed during periods of high load, and the observed behaviour on a free account is that a two-minute cron delivers roughly every three to five minutes and occasionally skips a slot entirely under Monday-morning load. Treat the effective minimum granularity as five minutes and the worst case as ten. There is no retry for a missed slot; the next run simply does more work.

That is fatal for anything that reads a deadline off the cron. Two things in this system have hard deadlines. Offer waves expire on `OFFER_WAVE_TTL_SECONDS = 600, 600, 900` for waves 1, 2 and 3: a wave that has lapsed must move to the next wave or to UNASSIGNED, and a fleet that answers after its wave closed must lose. The Advisor SLA is two hours from request to first expert response, and a breach is a promise broken to a farmer, not a metric.

So neither is implemented as an event the cron produces. Both are stored timestamps — `offer_expires_at` on job_offers, `sla_due_at` on advisory_requests — written at creation from the authoritative clock, and every read path evaluates them lazily. An offer fetched after its `offer_expires_at` is not returned as open, regardless of whether any sweep has run; an accept on it is 409 OFFER_EXPIRED, decided by comparing `now()` to the stored column inside the accepting transaction. An advisory request past `sla_due_at` is rendered as breached by the query that lists it. The cron then does only the work that lazy evaluation cannot do on its own: advancing a wave so the next set of fleets is notified, and raising the alert nobody is looking at a screen for.

THE RULE THIS GENERALISES TO

Correctness is computed from stored timestamps at read time. The cron exists to make things happen when no user is present — a notification, an accrual, an alert — and it is allowed to be late for that. If a rule would be wrong because a cron was five minutes late, the rule is in the wrong place. Every deadline in AA-03 therefore has a column, not a job.

WHAT THIS COSTS, AND THE PAID ALTERNATIVE

A fleet in wave 2 may not be notified for up to ten minutes after wave 1 lapsed, so allocation is slower than the TTLs imply and a shipment can sit unassigned longer than the 600+600+900 second budget suggests. The fix is a real scheduler: Cloud Scheduler is roughly 30 rupees a month for three jobs beyond the free three, or Cloud Run min-instances=1 with an in-process timer at roughly 1 100 rupees a month. Both break I-14. Neither is bought at MVP; AA-09 lists them as priced gaps with this section as the reason.

2.8 DERIVING ORDERS.STATUS

Invariant I-11: a projection of the shipment set, never a written field

Once an order splits into one shipment per supplier, orders.status stops being a fact and becomes a summary. Two shipments, one delivered and one still packing — the order is neither DELIVERED nor PACKED, it is whatever the reduction says it is. So the column is maintained as a projection: reduceOrderStatus() takes the shipment set and returns one of the ten statuses, the first matching condition wins, and evaluation order is the specification.

#	CONDITION OVER THE SHIPMENT SET — FIRST MATCH WINS	orders.status
1	All shipments CANCELLED	CANCELLED
2	All shipments DELIVERED	DELIVERED
3	All shipments terminal, mix of DELIVERED and RETURNED/RTO_DELIVERED	PARTIALLY_DELIVERED
4	All shipments RETURNED or RTO_DELIVERED	RETURNED
5	Any shipment in RETURN_REQUESTED..RETURN_PICKED	RETURN_IN_PROGRESS
6	Any shipment OUT_FOR_DELIVERY or FAILED_ATTEMPT	OUT_FOR_DELIVERY
7	Any shipment HANDED_OVER, IN_TRANSIT or RTO_INITIATED	DISPATCHED
8	Any shipment PACKING or READY_FOR_PICKUP	PACKED
9	Any shipment AWAITING_ACCEPTANCE or ACCEPTED	CONFIRMED
10	Payment not captured and not COD-confirmed	PENDING_PAYMENT

The terminal cases are tested first on purpose. Asking "is everything finished, and how" before asking "is anything in flight" means a fully delivered order can never be reported as OUT_FOR_DELIVERY because one shipment happens to sit in a stale state. The last row is the floor: if payment is neither captured nor COD-confirmed, nothing downstream matters and the order is PENDING_PAYMENT. The ten statuses in the app contract are PENDING_PAYMENT, CONFIRMED, PACKED, DISPATCHED, OUT_FOR_DELIVERY, DELIVERED, PARTIALLY_DELIVERED, CANCELLED, RETURN_IN_PROGRESS, RETURNED — the same set the app already parses, which is the constraint the whole design had to satisfy.

2.8.1 One pure function, one writer, one stream

reduceOrderStatus(shipments) is pure. No database access, no clock, no order row as input: it takes an array of shipment states and returns a status. That is what makes it testable as a table rather than as a scenario, and the unit suite drives all 28 reachable shipment-state combinations to an expected status. A pure reducer also means the answer cannot depend on which code path asked.

recomputeOrder(orderId) is the only writer of the column, and it lives in orders/fsm.ts. It loads the shipments, calls the reducer, and if the result differs from the stored value it writes the column and appends an order_events row in the same transaction. Every shipment transition calls it as its last step. Nothing else may.

The mobile tracker reads order_events rather than polling orders.status, which is what keeps the app contract frozen while the model underneath it changes completely. The app was already consuming that stream; it now sees more events in it, with shipment-level granularity available to a client that wants it and ignorable by a client that does not. An old build renders the same timeline it always did. This is section 2.2.1 in practice: the new capability went to a new field inside an existing append-only stream, not to a changed response shape.

2.8.2 The guard: extension chosen, trigger rejected

A convention that says "only recomputeOrder writes this column" is worth nothing in week 8 when somebody fixes a stuck order with a one-line update. Two mechanisms can enforce it and only one was chosen.

The Prisma client extension is the chosen guard. The same extension layer that does masking inspects every update and updateMany against the order model, and throws if the data object contains a status key unless an async-local-storage flag set by recomputeOrder is present on the call stack. It is paired with the ESLint rule aa/no-status-write, which rejects a status key inside any prisma update outside a file named fsm.ts, so the ordinary case fails at lint time and the clever case fails at runtime. Both live in the api repository, both are visible in a stack trace that names the file and line, and both are covered by the unit suite.

A Postgres trigger raising on any UPDATE of orders.status not originating from recomputeOrder was rejected, and it is the stronger control on paper — it survives psql, a migration script and a future second writer. It lost on identification. The trigger cannot see which application function issued the statement; it can only see a session, so it would need recomputeOrder to SET LOCAL a session variable that the trigger reads. That is the same pooled-connection fragility that ruled out row-level security in section 2.5, with the added failure mode that a missed SET LOCAL raises on a legitimate write and takes order transitions down in production rather than failing safe. A guard whose false-positive mode is an outage is worse than a guard whose false-negative mode is a lint bypass, at this team size.

The residual gap is stated rather than hidden: a manual UPDATE in the Neon console still succeeds. Nothing in this design prevents it, the audit trail will not carry it, and recomputeOrder will overwrite it on the next shipment transition anyway. AA-09 puts direct console writes under a runbook that requires a second person, which is a process control and is admitted as one.

WHY NOT DROP THE COLUMN AND COMPUTE ON READ

Deriving on every read would make the invariant structural rather than enforced, and that was the first design. It lost to querying: the admin order list filters and sorts by status across a table that grows without bound, and a computed status cannot be indexed. A denormalised column with one writer and two guards is the compromise, and order_events is the audit trail that makes a wrong value diagnosable after the fact.

2.9 ERRORS

Fourteen stable codes, and why the message is not one of them

CODE	HTTP	WHAT IT MEANS, AND WHAT THE CALLER SHOULD DO
ACTOR_MISMATCH	401	The token audience does not match the namespace (I-02). Returned instead of 403 so the existence of the route is not confirmed to the wrong audience. Do not refresh; the token is valid, it is simply not for this surface.
SCOPE_IN_REQUEST	400	The request carried partner_org_id or fleet_org_id (I-03). The client is scoping when it should not; remove the field. Logged as a security event.
PERMISSION_DENIED	403	The principal is on the right namespace and lacks the permission. The response names the permission required so the portal can render a useful empty state rather than a blank screen.
STEP_UP_REQUIRED	401	The route needs a TOTP factor verified within 300 seconds. Carries challenge: { factor: "totp", max_age_seconds: 300 } so the kit's step-up dialog can collect a code and replay the original request without losing form state.
IDEMPOTENCY_MISMATCH	409	The Idempotency-Key has been used on this route by this actor with a different body. The client's key derivation is wrong; do not retry with the same key.
OFFER_EXPIRED	409	offer_expires_at is in the past, decided against the stored column rather than against a sweep (section 2.7). The offer is gone; refresh the inbox.
OFFER_ALREADY_TAKEN	409	Another fleet accepted first. Distinct from OFFER_EXPIRED because the two mean different things to a dispatcher watching an acceptance rate.
DISCLOSURE_WINDOW_CLOSED	403	A manifest read outside the job window or outside MANIFEST_WINDOW_MINUTES = 240 (I-07). No address is returned and the attempt is registered.
LEDGER_IMMUTABLE	409	An UPDATE or DELETE was attempted against wallet_ledger, partner_payouts or fleet_payouts (I-04). Corrections are compensating rows. There is no override.
FLEET_BLOCKED_COD	409	The fleet org has an unreconciled COD period past its deadline, so it may not accept further COD offers. Clears when the remittance is reconciled.
STOCK_INSUFFICIENT	409	The requested quantity exceeds available stock at the moment of the transaction, checked inside it rather than before. Includes the available figure.

CODE	HTTP	WHAT IT MEANS, AND WHAT THE CALLER SHOULD DO
VALIDATION_FAILED	400	Zod rejected params, query or body. Carries a field path per error. On a portal this usually means a stale contract pin — see AA-01 section 1.11 failure mode 1.
NOT_FOUND	404	The row does not exist, or it exists in another tenant (section 2.5 rule 2). The two are deliberately indistinguishable.
RATE_LIMITED	429	The namespace or principal bucket is exhausted. Carries Retry-After. The portal backs off; it does not loop.

The code is the contract. Every one of these is a member of a union in `packages/shared/errors.ts`, exported through `@atavishaala/contract`, and adding a code means adding a line there and a row in AA-04 — which is what makes a missing entry in the portal error map a type error rather than a raw string on a dispatcher's screen. The message is not a contract: it may be reworded in any release, it is never parsed, and a portal that switches on message text is broken by definition. Copy lives in the kit's code-to-string map, so the same `OFFER_EXPIRED` reads as one sentence in the fleet portal and another in the admin console.

2.10 OBSERVABILITY AND LIMITS

Structured logs with holes in them, one error tracker, and four buckets

Every log line is JSON from a Pino child logger bound at step 2 of the chain, so a request id, an actor kind and a tenant id are on every line without a handler passing them. Cloud Run collects stdout into Cloud Logging, which is the whole logging stack: no agent, no shipper, nothing to keep running. The redaction paths are set once, at logger construction, because a redaction list applied per call site is a list that will be forgotten at the one call site that mattered.

```
// src/platform/log.ts – Pino redaction. censor is '[redacted]'; these paths never print.
redact: { remove: false, censor: '[redacted]', paths: [
  'req.headers.authorization',      'req.headers.cookie',      'req.headers["x-idempotency-key"]',
  'req.body.otp',                  'req.body.otp_hash',      'req.body.totp',
  'req.body.phone',               'req.body.email',         'req.body.address',
  'req.body.account_number',      'req.body.ifsc',          'req.body.upi_handle',
  'req.body.gstin',              'req.body.pan',           'req.body.aadhaar',
  'req.body.licence_number',      'req.body.reg_number',    'req.body.password',
  '*.otp', '*.otp_hash', '*.phone', '*.account_number', '*.aadhaar', // nested wildcards
  'res.body.manifest.address',     'err.config.headers',     'err.request.body',
] }
// OTP values are bcrypt hashes in the database and are never logged in either form.
// A wildcard path costs a tree walk per line; the flat paths above are listed explicitly
// so the hot request-log path does not pay for the nested ones.
```

Sentry is the error tracker, free tier, with a `beforeSend` hook that runs the same masking helper the serialiser uses over the event's request body, headers, breadcrumbs and extra context, drops the event entirely if it cannot parse what it is scrubbing, and strips the local variable capture that would otherwise carry a phone number out of a stack frame. The request id is set as a Sentry tag, which is the join key between an exception and the log lines around it: `X-Request-Id` is echoed on every response and rendered in the portal error toast, so a support conversation produces an id that finds both.

NAMESPACE	BUCKET	LIMIT	REASON FOR THIS NUMBER
/api/v1/public	IP	30 / min	OTP request is here. Tight enough to make phone enumeration expensive, loose enough for a shared NAT in a village.
/api/v1/app	user id	120 / min	Existing, unchanged. A browsing customer bursts.
/api/v1/admin	staff id	240 / min	Highest, on purpose: a dispatcher with four list views open and 30-second polling is a legitimate heavy caller.
/api/v1/partner	partner_org_id	120 / min	Per organisation, not per user, so one supplier cannot spend another's budget.
/api/v1/fleet	fleet_org_id	180 / min	Per organisation. A dispatch screen plus riders on mobile networks retry more than a desk user does.

The bucket is a Postgres row per (bucket key, minute), incremented in its own statement before the handler runs, because there is no Redis and adding one would break I-14. It is therefore a coarse limiter: it counts per minute rather than sliding, and it costs one write per request. That is the accepted price of not paying for a cache.

WHAT IS NOT INSTRUMENTED, AND WHY

There is no APM, no distributed tracing, no metrics backend and no dashboard. Cloud Trace would cost nothing at this volume but adds a dependency and a sampling decision for a single-process system where the request id already tells the whole story. There is no uptime SLO and no error-rate alert: UptimeRobot pings the health route every five minutes and that is the entire alerting surface, so a 30 per cent failure rate that does not take the process down is invisible until somebody reads Sentry. Latency is knowable only from Cloud Run request logs after the fact. The honest statement is that this system is diagnosable but not observable — it answers "what happened to this request" well and "is the system healthy right now" badly. AA-09 names the paid gaps: a metrics backend at roughly 800 rupees a month, and Sentry performance monitoring beyond the free error quota.

2.11 TEST STRATEGY

Three shapes of test, and the ones a merge cannot happen without

Coverage percentage is not a target and is not measured. What is measured is whether each invariant has a test that fails when the invariant is broken, which is a different and much smaller set. The suite has three shapes and each one exists for a reason a reviewer could not cover by reading.

SHAPE	RUNNER AND SCOPE	WHAT IT IS FOR
Unit, on pure functions	Vitest, no database. reduceOrderStatus over all 28 shipment-state combinations as a table; the offer allocator over wave composition, rate-card ties and the eight-job rider cap; paise arithmetic and rounding.	These two functions carry the highest ratio of branch count to line count in the codebase and both are pure, so a table test is exhaustive rather than illustrative. A failure here is a wrong answer, not a wrong wiring, and it is found in two seconds without a database.
Integration, on the router	Vitest against a Neon branch seeded with two partner orgs, two fleet orgs and a staff user per role. Real Express, real Prisma, real middleware chain.	The scoped() denial suite lives here and it is the most important file in the repository: for every tenant-owned model, org A requests org B's row by id and must receive 404, and org A lists and must not see one org B row. Also every cross-audience call asserting 401 ACTOR_MISMATCH, every step-up route asserting 401 STEP_UP_REQUIRED without a fresh factor, and the SCOPE_IN_REQUEST rejection on body, query and path.
Contract, against fixtures	Recorded request/response fixtures for every /api/v1/app endpoint, compared byte for byte, plus an idempotency replay of all 70 keyed routes asserting the second response is identical to the first.	This is what makes section 2.2.1 real. The app contract cannot be frozen by intention; it is frozen by a test that fails on a reordered key or an added field. The replay half is what makes I-12 real for the same reason: a replay that returns a different id passes a status-code assertion and fails this one.

2.11.1 The test each invariant needs before merge

Ten of the fourteen invariants are enforceable inside this repository. For each one, the named test below must exist and be red against a deliberately broken build before the pull request that introduces the mechanism is merged. Writing the test first is not the requirement; watching it fail is, because a test that has never failed is a test nobody knows is wired up.

ID	INVARIANT	THE TEST THAT MUST EXIST
I-02	Namespace equals audience	A sweep over the route table calling every non-public route with the three wrong audiences. Asserts 401 ACTOR_MISMATCH and, separately, that no audit row was written.
I-03	Tenant scope comes from the token	partner_org_id in body, in query and in path, on three routes each, all asserting 400 SCOPE_IN_REQUEST and a security log line. Plus a lint fixture proving aa/no-tenant-in-input rejects the schema.
I-04	Money is integer paise on an append-only ledger	An UPDATE and a DELETE attempted against each of the three ledger tables, asserting the Postgres rule raises; a schema grep asserting no *_paise column is FLOAT, DOUBLE, DECIMAL or NUMERIC.
I-05	Every state change goes through assertTransition	For each of the eight state machines, one legal transition asserted to succeed and one illegal transition asserted to raise, driven from the same transition table the implementation reads.
I-06	Admin surfaces show the last four characters only	A serialiser test over every response schema carrying a PII field, asserting the emitted string is four characters. Plus a type-level assertion that the Permission union contains no member matching /raw unmask reveal/.
I-07	Delivery address release is job-scoped, window-scoped and logged	Manifest read inside the window returns an address and writes exactly one pii_disclosures row; outside the window and outside the job states returns 403 DISCLOSURE_WINDOW_CLOSED; the phone field is absent in all cases.
I-08	Every admin write is audited with a before/after diff	A fixture per admin mutation asserting audit_log grows by exactly one row carrying actor, permission, target and a non-empty diff, with the middleware mounted at the router so a new route inherits it.
I-09	Staff authentication requires TOTP; finance actions re-prompt	Login without an enrolled factor is refused before a session is issued; each step-up route is called with a factor verified 301 seconds ago and asserts 401 STEP_UP_REQUIRED.

ID	INVARIANT	THE TEST THAT MUST EXIST
I-11	Customer order status is derived, never written	The 28-combination reduction table, plus an attempted prisma update writing status outside fsm.ts asserting the extension throws, plus a lint fixture for aa/no-status-write.
I-12	Idempotency keys on money and on job offers	A replay of all 70 keyed routes asserting byte equality, a mismatched body asserting 409 IDEMPOTENCY_MISMATCH, and two concurrent first attempts asserting exactly one write.

The remaining four are not testable here. I-01, I-10 and I-13 are properties of the portal repositories and are covered by the gates in AA-01 section 1.7; I-14 is a budget reviewed at the Friday gate and is a ledger, not an assertion. Of the twelve CI gates, six run in this repository: typecheck, lint including the eight custom rules, unit and integration tests, the contract fixture sweep, the permission sweep and the idempotency replay, with a migration rehearsal against a restored branch added on any pull request touching prisma/migrations.

WHAT THIS DOCUMENT DOES NOT COVER

The tables, columns and the eight migrations are in AA-03. The endpoint list with permissions and request shapes is in AA-04. The permission matrix, the token and cookie posture and the full PII disclosure contract are in AA-08. Cron operations, alert thresholds and the priced gaps named in sections 2.7 and 2.10 are in AA-09. The contract and kit packages this repository publishes into are in AA-01. The week-by-week sequence that builds the eleven modules is in AA-11.